

AI 前端开发常见错误与避坑指南

适用平台：所有 AI 编程工具 核心原则：把 AI 当"过度自信且容易犯错的初级开发者"对待，始终 review 和测试 参考来源：Addy Osmani 博客、火山引擎社区、腾讯云开发者社区、多位实践者经验

一、提示词层面的常见错误

错误类型	表现	解决方案
模糊意图	"帮我写个登录页"	明确技术栈、UI 风格、功能细节
缺少约束	AI 随意引入第三方库	在 Prompt 中明确禁止
一次要求太多	AI 生成混乱代码	分步、分功能逐步添加
忽略状态设计	只考虑正常状态	强制要求 Loading / Error / Empty 三种状态
未指定响应式	移动端样式错乱	明确断点和移动优先策略

二、工作流层面的常见错误

1. 跳过规划直接写代码

- 错误：**直接用模糊 Prompt 让 AI 生成代码
- 正确：**先用 Prompt 让 AI 帮你做需求分析和架构设计，生成 spec.md 后再开始编码
- 来源：**Addy Osmani（Google Chrome 团队前负责人）

2. 大文件不拆分

- 错误：**让 AI 不断往一个文件里加代码
- 正确：**在 Prompt 中植入"治理指令"

系统规则：当任一文件超过 250 行，或包含多个独立的逻辑块时，请主动询问我是否需要拆分为子组件。

3. 不使用版本控制

- **错误**: AI 改坏了代码无法回滚
- **正确**: 在写第一行代码前初始化 Git, 每完成一小步就 commit

4. 盲目信任 AI 输出

- **错误**: 直接使用 AI 生成的代码不审查
- **正确**: 必须经过"AI 生成 → 逐行审查 → 理解逻辑 → 运行测试 → 提交"

5. 让 AI 同时处理前后端

- **错误**: 一次 Prompt 要求 AI 同时写前后端
- **正确**: 分层解耦 — 先前端 (Mock 数据) → 再后端 → 最后集成

三、AI 前端生成的常见视觉问题

1. "紫色审美"

LLM 生成的 UI 倾向于使用紫色系配色

解决: 在 Prompt 中明确指定色彩方案和色值

2. 过度动画

AI 默认添加 bouncy easing 等动画

解决: 在 Prompt 中为动效系统定调, 明确 "Fast, snappy, mechanical"

3. 响应式适配不足

切换到移动端时比例失衡或跑版

解决: 在 Prompt 中定义完整的响应式策略, 包括 7 条适配规则

4. 忽视无障碍

缺少 ARIA 标签、焦点状态、键盘导航

解决: 在 Prompt 中强制要求 WCAG 2.1 AA 合规

5. 所有页面看起来一样

AI 生成的页面缺乏品牌个性

解决：使用"对标法"+"排除清单"定义独特风格

四、避坑检查清单

写 Prompt 前检查

- ☐ 是否明确了技术栈？
- ☐ 是否列出了具体功能点？
- ☐ 是否定义了视觉风格（色值 / 字体 / 布局）？
- ☐ 是否定义了交互行为（动效 / 状态切换）？
- ☐ 是否列出了排除清单？
- ☐ 是否要求了三种 UI 状态（Loading / Error / Empty）？
- ☐ 是否指定了响应式策略？
- ☐ 是否要求了无障碍支持？

使用 AI 时检查

- ☐ 是否分步、分功能进行？
- ☐ 是否使用了版本控制（Git）？
- ☐ 是否审阅了 AI 生成的每一行代码？
- ☐ 是否在真实设备上测试了？
- ☐ 是否检查了性能（动画帧率 / 加载速度）？

五、核心方法论补充

"前端决定后端"原则

原则：先画出界面，用 Mock 数据跑通交互。
UI 怎么长，决定了后端数据库怎么建。

具体做法：

1. 先在前端代码中编写 Mock Data（实质是定义数据契约）
2. Mock Data 的 TypeScript 接口定义就是后端 API 的规格说明书

3. 后端实现时严格遵循前端定义的数据结构

"UI 状态完备性"要求

拒绝"仅演示可用"。强制要求 AI 实现三种状态的 UI：

- Loading（加载中）
- Error（加载失败）
- Empty（无数据）

在 Prompt 中加入：

每个列表/数据展示组件必须包含：

1. 数据加载时的骨架屏或 Spinner
2. 请求失败时的错误提示和重试按钮
3. 数据为空时的空状态插图和引导文案

"渐进式提示词路径"

复杂应用不要一步到位，采用分层构建：

路径 1：先构建核心功能（可用的 MVP）

路径 2：增加次要功能（社交分享、通知等）

路径 3：增加高级功能（奖励机制、数据分析等）

微提示词调整的类型：

- 做加法："添加评论区"
- 做改变："颜色改成炭灰色"
- 找参考："像 Notion 的风格"

规则文件方法论

将项目规范持久化为规则文件，AI 每次会话自动加载：

CLAUDE.md / .cursorrules 文件内容建议：

1. 项目编码风格（缩进、命名规范）
2. 技术栈约束（禁止随意引入新库）
3. 组件模式（函数式 vs 类式）
4. 状态管理方案
5. CSS 方案（Tailwind 配置）
6. 测试要求
7. 反幻觉规则："如果不确定，请询问而非编造"

模型选择与并行验证策略

1. 不同模型有不同的"个性"，如果一个模型卡住，换另一个试试
2. 用一个模型写代码，用另一个模型做 `code review`
3. 选择最新、最强的模型版本（质量差异很大）
4. 个人偏好很重要 – 选一个"感觉对"的模型

六、参考来源

来源	核心贡献
Addy Osmani - LLM Coding Workflow 2026	workflow避坑、规则文件方法论
火山引擎 - Vibe Coding 工程化最佳实践	分层解耦、前端决定后端
腾讯云 - AI 提示词最佳实践	全场景 Prompt 检查清单